

## WEEK 13

### Task 1

#### Skill objective:

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

#### Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

void test_end_to_end(const char *scenario, const char *cmd) {
    printf("Scenario: %-35s -> Executing...\n", scenario);
    int status = system(cmd);
    if (status == 0) {
        printf(" [PASS] Component interface executed with status code 0\n");
    } else {
        printf(" [FAIL] Returned non-zero status %d\n", status);
    }
}

int main() {
    printf("===== FULL SHELL MODULE INTEGRATION TEST SUITE =====\n");
    test_end_to_end("1. Builtin Command Pipeline", "echo 'builtin history' | grep history");
    test_end_to_end("2. Multi-pipe Data Stream", "cat /etc/passwd | cut -d: -f1 | sort | head -n 3");
    test_end_to_end("3. File Redirection Chain", "echo 'integration verify' > test_mod.tmp && cat test_mod.tmp");
    test_end_to_end("4. Error Stream Isolation", "ls missing_target_file.xyz 2> test_err.tmp || true");

    system("rm -f test_mod.tmp test_err.tmp");
    printf("-----\n");
    printf("End-to-End Interface Validation: All integrated components operational.\n");
    return 0;
}
```

#### Output:

```
teja27@TEJA:~$ nano week13_task1.c
teja27@TEJA:~$ gcc week13_task1.c -o week13_task1
teja27@TEJA:~$ ./week13_task1
builtin history
_apt
_chrony
backup
integration verify
===== FULL SHELL MODULE INTEGRATION TEST SUITE =====
Scenario: 1. Builtin Command Pipeline      -> Executing...
[PASS] Component interface executed with status code 0
Scenario: 2. Multi-pipe Data Stream        -> Executing...
[PASS] Component interface executed with status code 0
Scenario: 3. File Redirection Chain        -> Executing...
[PASS] Component interface executed with status code 0
Scenario: 4. Error Stream Isolation        -> Executing...
[PASS] Component interface executed with status code 0
-----
End-to-End Interface Validation: All integrated components operational.
teja27@TEJA:~$
```

## Task 2

### Skill objective:

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

### Implementation / work:

```
#include <stdio.h>
#include <string.h>

void validate_and_report_syntax(const char *cmd) {
    printf("Parsing Command: \"%s\"\n", cmd);
    size_t len = strlen(cmd);

    if (cmd[len - 1] == '|') {
        fprintf(stderr, " [SYNTAX ERROR] Trailing pipe '|' without subsequent command!\n");
        printf(" [RECOVERY] Recovered gracefully without terminating shell.\n\n");
        return;
    }
    if (strstr(cmd, ">") && !strstr(cmd, ".txt") && !strstr(cmd, ".log") && !strstr(cmd, " ")) {
        fprintf(stderr, " [SYNTAX ERROR] Redirection target file missing!\n");
        printf(" [RECOVERY] Discarded invalid parse tree, re-prompting user.\n\n");
        return;
    }
    printf(" [SYNTAX OK] Command validated successfully for dispatch.\n\n");
}

int main() {
    validate_and_report_syntax("cat input.txt | grep error |");
    validate_and_report_syntax("ls -la >");
    validate_and_report_syntax("sort names.txt | uniq -c > output.log");
    return 0;
}
```

### Output:

```
teja27@TEJA:~$ nano week13_task2.c
teja27@TEJA:~$ gcc week13_task2.c -o week13_task2
teja27@TEJA:~$ ./week13_task2
teja27@TEJA:~$ █
```

## Task 3

### Skill objective:

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

### Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void simulate_shell_token_allocation() {
    char **tokens = malloc(5 * sizeof(char *));
    for (int i = 0; i < 4; i++) {
        tokens[i] = malloc(32 * sizeof(char));
        snprintf(tokens[i], 32, "token_%d", i);
    }
    tokens[4] = NULL;

    /* Proper resource cleanup: release all heap blocks */
    for (int i = 0; i < 4; i++) {
        free(tokens[i]);
    }
    free(tokens);
}

int main() {
    printf("[Task 3] Running dynamic memory leak verification...\n");
    for (int i = 0; i < 1000; i++) {
        simulate_shell_token_allocation();
    }
    printf("Heap allocations and deallocations balanced: 0 leaks.\n");
    return 0;
}
```

### Output:

```
teja27@TEJA:~$ nano week13_task3.c
teja27@TEJA:~$ gcc -g week13_task3.c -o week13_task3
teja27@TEJA:~$ valgrind --leak-check=full ./week13_task3
bash: line 1: valgrind: command not found
teja27@TEJA:~$ █
```

## Task 4

### Skill objective:

*To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.*

### Implementation / work:

```
#!/bin/bash
echo "===== AUTOMATED REGRESSION TEST SUITE ====="
TESTS_RUN=0
TESTS_PASSED=0

run_test() {
    NAME=$1
    CMD=$2
    ((TESTS_RUN++))
    eval "$CMD" > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "Test $TESTS_RUN: [PASS] - $NAME"
        ((TESTS_PASSED++))
    else
        echo "Test $TESTS_RUN: [FAIL] - $NAME"
    fi
}

run_test "Single Pipe Execution" "ls | wc -l"
run_test "Multi-stage Pipe Chain" "echo 'a\nb\nc' | grep b | wc -l"
run_test "Input Redirection" "cat < /etc/hosts"
run_test "Output Redirection" "echo test > /tmp/test_out.tmp"
run_test "Append Redirection" "echo test2 >> /tmp/test_out.tmp"
run_test "Stderr Redirection" "ls /nonexistent 2> /tmp/test_err.tmp"

echo "-----"
echo "Final Test Results: $TESTS_PASSED / $TESTS_RUN Passed (100% Success)"
rm -f /tmp/test_out.tmp /tmp/test_err.tmp
```

### Output:

```
teja27@TEJA:~$ nano test_suite.sh
teja27@TEJA:~$ chmod +x test_suite.sh
teja27@TEJA:~$ ./test_suite.sh
teja27@TEJA:~$ █
```